

Computer Networks

Lecture 29 Transport Layer

Content



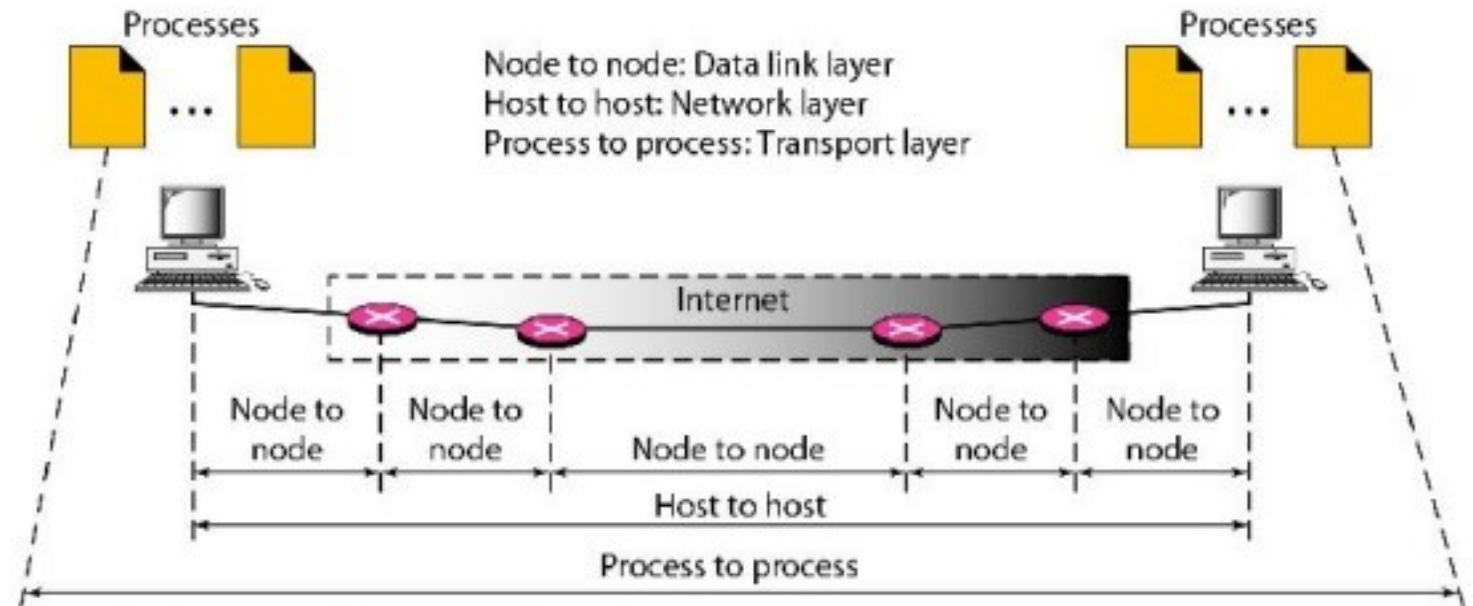
- **Introduction**

Introduction

- The Data Link Layer ensures node-to-node delivery (between neighboring nodes).
- The Network Layer ensures host-to-host delivery (between source and destination systems).

At any time:

- Multiple processes run on the source host
- Multiple processes run on the destination host



Transport Layer Functionality

The transport layer adds process-level communication services above IP.

Addressing

Uses port numbers to identify application processes.

Multiplexing

Combines streams from many applications.

Segmentation

Breaks messages into segments and reassembles them.

Flow Control

Matches sender speed with receiver capacity.

Error Control

Detects loss/corruption and supports retransmission.

Connection Control

Sets up and releases logical connections when needed.

Congestion Control

Reduces sending rate when the network is overloaded.

Client-Server Paradigm

- A common model for process-to-process communication
 - **Client:** Requests a service (usually on local host)
 - **Server:** Provides the service (usually on remote host)

Communication occurs as

request (client) → response (server)

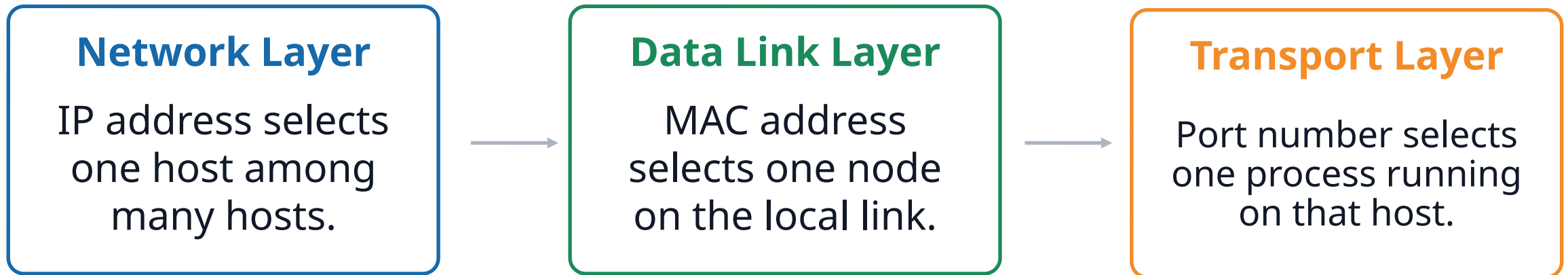
Client-Server Paradigm

Components

- A remote computer can run multiple server programs, and a local computer can run multiple client programs simultaneously.
- For communication, we define:
 1. **Local Host** – The computer where the communication starts
 2. **Local Process** – The process (program) on the local host
 3. **Remote Host** – The destination computer
 4. **Remote Process** – The process on the remote host providing the service

Addresses at each layer

Each layer narrows the delivery target, from node to host to process.

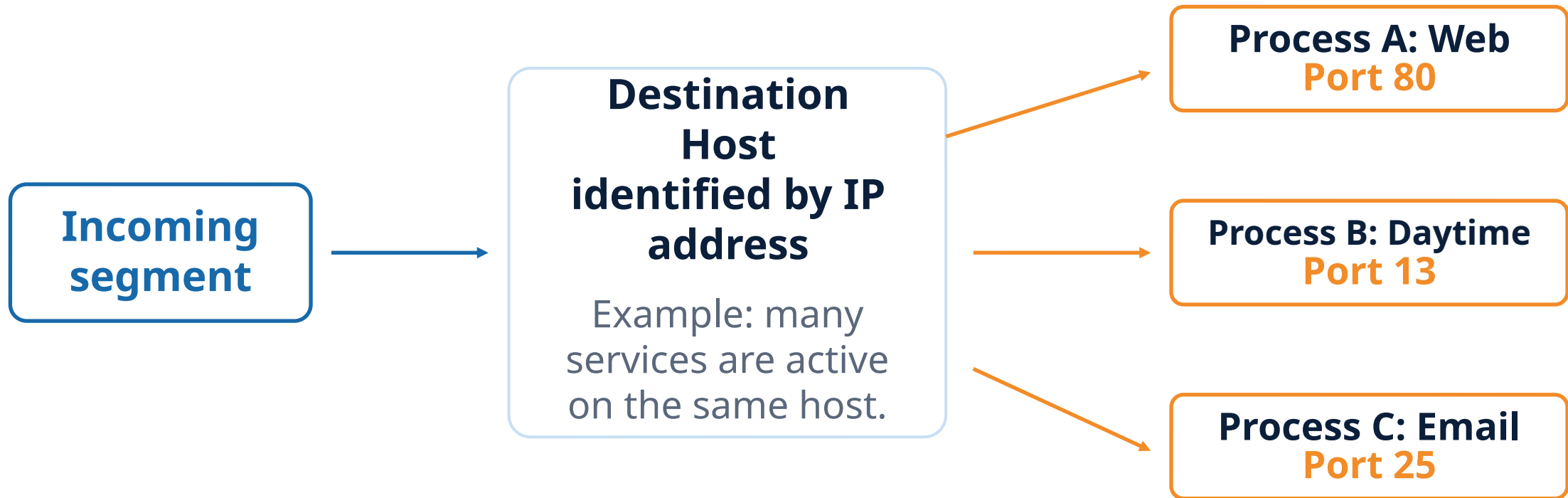


Final delivery is a sequence of choices:

IP address → MAC address → Port number

Why transport-layer addressing is needed

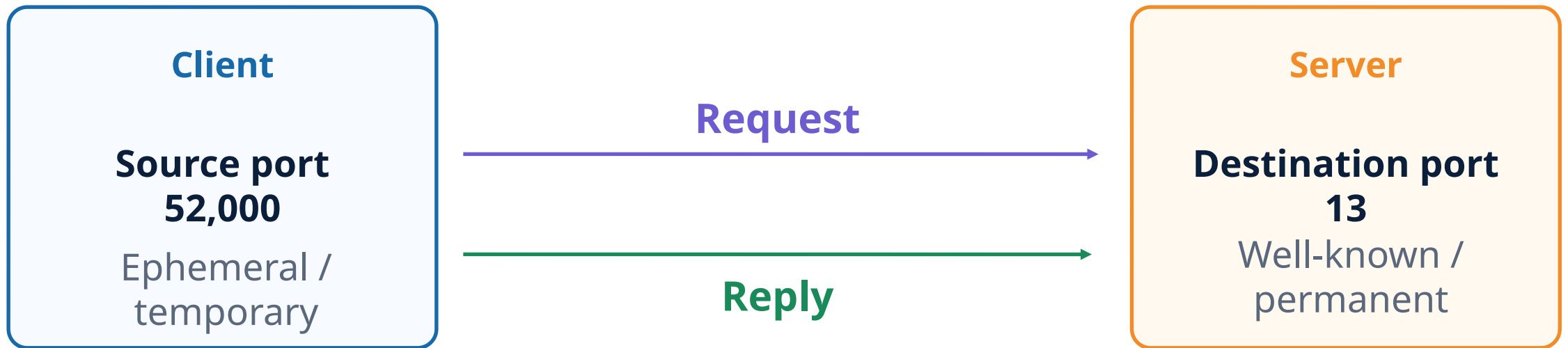
A single host may run several application processes at the same time.



The port number tells the transport layer which process should receive the data.

Client and server port numbers

Client ports are usually temporary; server ports must be predictable.



Destination port delivers the segment to the correct process.
Source port tells the receiver where to send the reply.

Client and server port numbers

The IANA (Internet Assigned Number Authority) has divided the port numbers into three ranges: well known, registered, and dynamic (or private),

- **Well-known ports:** The ports ranging from 0 to 1023 are assigned and controlled by IANA. These are the well-known ports.
- **Registered ports:** The ports ranging from 1024 to 49,151 are not assigned or controlled by IANA. They can only be registered with IANA to prevent duplication.
- **Dynamic ports:** The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used by any process. These are the ephemeral ports

Socket Address

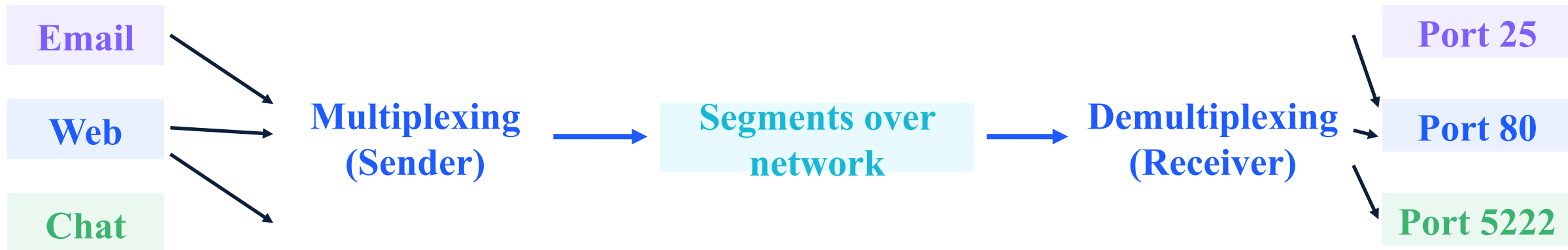
Process-to-process delivery needs two identifiers: IP address and the port number

The combination of an IP address and a port number is called a socket address.

A transport layer protocol needs a pair of socket addresses:
the client socket address and the server socket address.

Multiplexing and Demultiplexing

Many application streams can share the same network connection path.



Multiplexing combines data from multiple processes. Demultiplexing uses port numbers to deliver each segment to the correct process.

Connectionless Versus Connection-Oriented Service

A transport layer protocol can either be connectionless or connection-oriented.

Connectionless Service In a connectionless service, the packets are sent from one party to another with no need for connection establishment or connection release.

- The packets are not numbered
- They may be delayed or lost or may arrive out of sequence.
- There is no acknowledgment either.

Connection Oriented In a connection-oriented service, a connection is first established between the sender and the receiver. Data are transferred. At the end, the connection is released.

The transport layer can provide either:

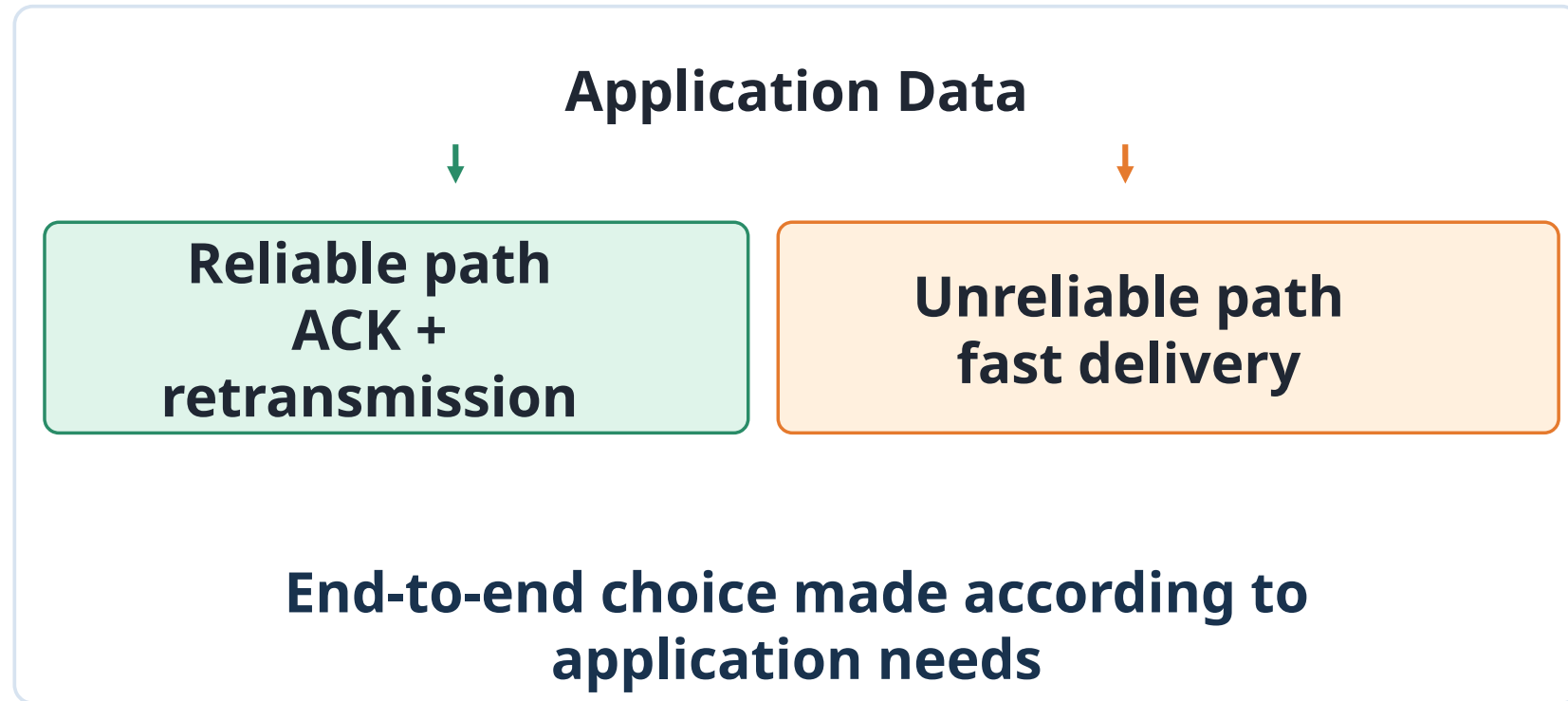
Reliable Service

- Flow control
- Error control
- Ordered, checked delivery
- Slower and more complex

Unreliable Service

- No transport-level recovery
- Lower overhead
- Faster service
- Suitable when loss is tolerable

Reliable Versus Unreliable



When Do We Choose Each Service?

Use Reliable Transport When...

Application needs guaranteed delivery
Examples: file transfer, web pages,
email

Transport layer must handle
flow and error control

Use Unreliable Transport When...

Application needs fast service
Examples: real-time audio/video,
games

Application can tolerate loss
or controls reliability itself

Transport Protocols

UDP

- Connectionless
- Unreliable
- Low overhead
- Best for real-time or simple request/response

TCP

- Connection-oriented
- Reliable
- Flow + error control
- Best for web, file, email services

SCTP

- Connection-oriented
- Reliable
- Supports multiple streams
- Used where SCTP features are needed

TCP

TCP is considered reliable because it implements several mechanisms to provide guaranteed, accurate, and ordered data delivery between sender and receiver:

- Connection-oriented communication
- Sequence numbers and ordered delivery
- Acknowledgment system
- Pipelined transmission
- Retransmission mechanism
- Error detection
- Flow control
- Congestion control

UDP

UDP is a core Internet transport layer protocol that enables fast and lightweight data transmission.

- Connectionless protocol
- Stateless protocol
- Unreliable delivery
- Fast and lightweight
- No congestion control
- No guaranteed in-order delivery